TRIBHUVAN UNIVERSITY

INSTITUTE OF SCIENCE AND TECHNOLOGY

KIRTIPUR, NEPAL

CASE STUDY : FOODMANDU

OBJECT ORIENTED SOFTWARE ENGINEERING

SUBMITTED BY:

PRASHANT BHANDARI

ROLL NUMBER: 41(B)

SUBMITTED TO:

PROF. DR. SUBARNA SHAKYA

CENTRAL DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

Date: May, 12 2025

1. Introduction

Foodmandu is a pioneering food delivery platform in Nepal, transforming how people order food since its launch in 2010. By connecting customers with diverse restaurants through its user-friendly mobile app and website, Foodmandu has redefined convenience in the food industry. This case study examines how Foodmandu applies Object-Oriented Software Engineering (OOSE) principles to build a scalable, efficient, and maintainable system. From class hierarchies to modular design, OOSE is crucial in structuring Foodmandu's platform, ensuring seamless operations and a smooth user experience. Understanding these technical foundations sheds light on the system's reliability and adaptability in a growing market.

2. Objectives of the Case Study

- To understand how Foodmandu applies object-oriented principles in its software system.
- To identify the requirement, analysis, design, and implementation models in its development lifecycle.
- To evaluate how OOSE contributes to the scalability, maintainability, and reliability of the platform.

3. Description about the Case Study Area

The focus of this study is to demonstrate the software engineering approach used by Foodmandu to efficiently manage and streamline various key operations such as:

- Customer ordering processes
- Restaurant management
- Delivery tracking
- Real-time notifications
- User accounts and wallets

These core functionalities are accessed through both web and mobile platforms, offering users a seamless experience. The system integrates features like digital payments, GPS tracking, and customer review mechanisms, enabling Foodmandu to deliver reliable and responsive service in real time.

4. The Requirement Model

The requirement model for Foodmandu outlines both functional and non-functional aspects of the system, which are essential to provide a seamless and reliable food delivery experience. These requirements were identified during the early stages of system development to ensure that the application meets the expectations of users, restaurant partners, and administrators.

Functional Requirements:

These define the core actions the system must support to function properly:

- User Registration/Login
- Browse restaurants and food items
- Place, cancel, and track orders
- Secure payment system
- Restaurant panel for order acceptance
- Admin dashboard

Each functional requirement is mapped to specific user roles such as customers, restaurant staff, and administrators, to ensure targeted access and control. These features enable smooth user interaction and effective management of food orders and delivery.

Non-Functional Requirements:

These focus on the quality and performance of the system:

- High availability
- Secure transactions
- Fast response time
- Scalable architecture
- Real-time order status

These non-functional attributes ensure that the platform remains efficient, secure, and scalable even as user demand increases. For instance, high availability guarantees minimal downtime, while real-time order status enhances user satisfaction. Together, these requirements form the foundation of a robust, object-oriented food delivery system.

5. The Analysis Model

Using Object-Oriented Analysis, the following key classes were identified:

- **User Class:** Attributes include name, email, location. Behaviors include register(), login(), placeOrder().
- **Restaurant Class:** Attributes include restaurantName, menu, rating. Behaviors: addItem(), acceptOrder().
- **Order Class:** Attributes include orderID, items, totalAmount. Methods: calculateTotal(), updateStatus().
- **FoodItem class:** Food item, price, and quantity.
- **Delivery Class:** Tracks delivery status and estimated time.

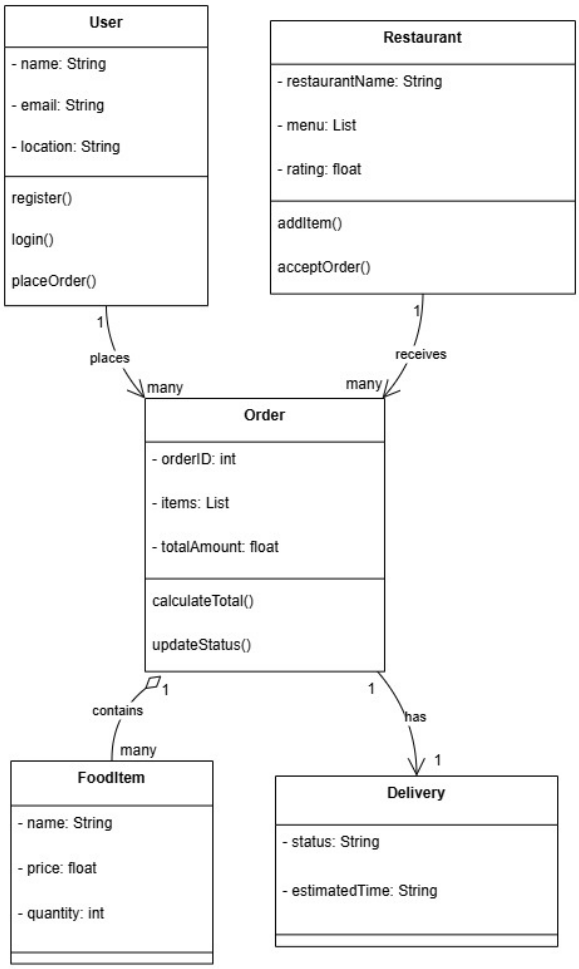

Figure 1: Class Diagram for the Analysis Model

6. The Design Model

UML Design Patterns Used:

- **MVC (Model-View-Controller):** Separates the interface, data processing, and business logic, allowing better maintainability and scalability of the codebase.
- **Observer Pattern:** Used for real-time order tracking and notification, ensuring customers and delivery staff receive updates as events occur.
- **Factory Pattern:** Used for creating different user types (Admin, Customer, Delivery Staff), making the system flexible and easily extendable.

System Architecture Design

The architecture diagram illustrates Foodmandu's streamlined three-tier system design. At the front-end, users including customers, restaurants, and delivery personnel interact through responsive web and mobile interfaces. These interfaces connect to a robust API layer that manages all request-response cycles and data exchange. The core business logic layer handles critical operations like order processing, payment transactions, and delivery management. This clear separation of concerns between presentation (UI), application (API), and business logic layers enables the platform to scale efficiently while maintaining optimal performance.

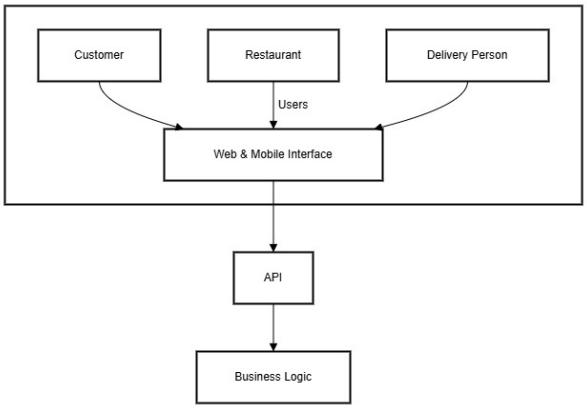

Figure 2: System Architecture Design

Sequence Diagram

This sequence diagram illustrates Foodmandu's end-to-end order processing system.

- **Restaurant** accepts/rejects the order.
- **Payment** is processed via gateway.
- **Delivery** is assigned (accepted/rejected) with GPS tracking.

- **Notifications** update status to customer/restaurant.
- **Rejected** orders trigger reassignment.

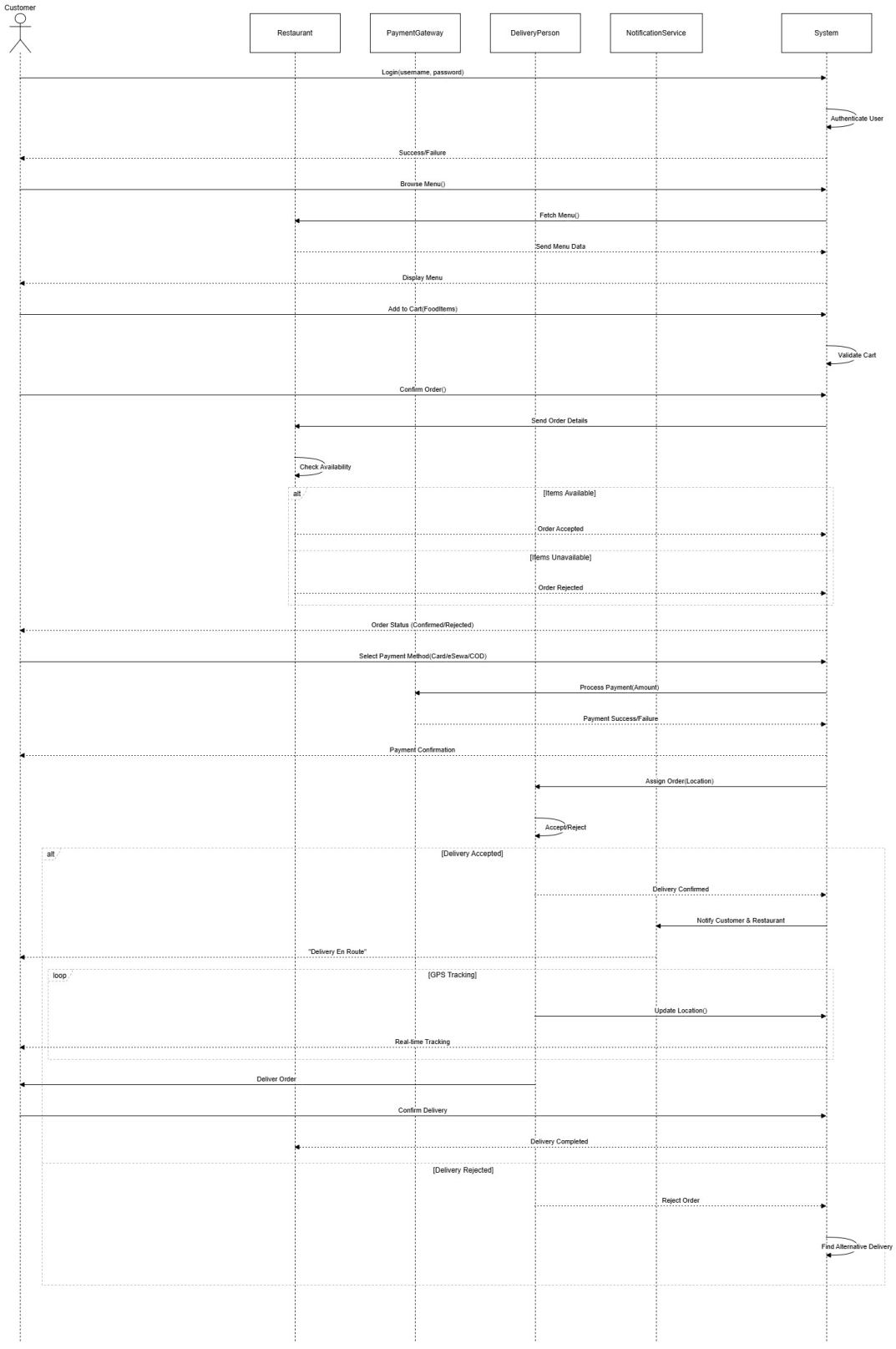

Figure 3: Sequence Diagram for the Design Model

7. Implementation Model

Foodmandu's system is architected to provide a seamless and efficient food delivery experience, leveraging a combination of modern technologies and design principles.

Frontend Development

- **Mobile Applications:** Developed using Kotlin and Java for Android platforms, and Swift and Objective-C for iOS platforms. These native applications ensure optimal performance and integration with platform-specific features. Ekbana Solutions
- **Web Interface:** While specific frameworks aren't detailed in the available sources, modern web development practices suggest the use of responsive and interactive frameworks to enhance user experience.

Backend Development

- **Programming Languages:** Utilizes Node.js and Python (with frameworks like Django or Flask) to handle server-side operations, business logic, and API integrations.
- **Architectural Patterns:** Employs Model-View-Presenter (MVP) and Model-View-Controller (MVC) design patterns to separate concerns, enhance code maintainability, and facilitate scalable development. Ekbana Solutions

Database Management

- **Databases:** Implements Realm for mobile data storage, providing an efficient and reactive database solution for mobile applications. 6sense

API and Third-Party Integrations

- **Payment Gateways:** Integrates with payment services such as Khalti and eSewa through their respective Software Development Kits(SDK), enabling secure and convenient transactions for users. Ekbana Solutions
- **Other Services:** Incorporates services like Google Services for functionalities such as location tracking and Facebook Software Development Kits(SDK) for social integrations.

8. Conclusion and Recommendation

Foodmandu's success is strongly rooted in its effective use of Object-Oriented Software Engineering (OOSE) principles. The platform effectively handles thousands of concurrent users,

supports complex food ordering workflows, and delivers a responsive and intuitive user experience. This is achieved through a modular, scalable, and object-based architecture that promotes code reusability, maintainability, and easier debugging. By incorporating proven design patterns such as MVC, and leveraging modern development tools, Foodmandu sets a benchmark for scalable, user-centric digital platforms.

Recommendation

- Enhance AI-based recommendations for personalized food suggestions.
- Improve Security with OAuth 2.0 for authentication.
- Optimize Delivery Routing.